Unité 4 — Défenses & contournements

Livrable B — Support de cours · UE « Software Security »

Code UE 7532 14 U32 D1 · 11 octobre 2026

> **Place dans le programme.** « Évaluer les bonnes pratiques et mécanismes de défense » et « identifier les limites des défenses et les moyens de les contourner ». On réactive ici les protections désactivées en Unité 2 et on étudie la course offensive/défensive. *Fondé sur meltypot (Buffer Overflow Defenses : canary/StackShield, libsafe, ASLR, pile non exécutable, ret-to-libc). ROP/ROP-chain/PIE/RELRO détaillés = contenu auteur.*

1. La course au durcissement

Chaque défense a suscité un contournement, qui a suscité une nouvelle défense. On présente les quatre protections majeures, leurs limites, et les techniques qui les contournent.

2. Stack canary

Le compilateur insère une valeur aléatoire (le **canari**) entre les variables locales et l'adresse de retour ; à la sortie de fonction, il vérifie qu'elle est intacte. Un débordement qui écrase l'adresse de retour écrase d'abord le canari → l'écart est détecté et le programme **abandonne**.

RÉEL — STACK3 RECOMPILÉ AVEC -FSTACK-PROTECTOR-ALL

```
$ ./stack3_canary "$(python3 -c 'print("A"*80)')"
*** stack smashing detected ***: terminated
exit=134  (SIGABRT)
```

Limites / contournements : fuite du canari (lecture mémoire, format string) puis réécriture à l'identique ; écrasement qui saute par-dessus le canari (pointeurs, indices) ; absence de canari sur certaines fonctions.

3. NX / DEP (pile non exécutable)

La pile (et le tas) sont marqués **non exécutables** : le shellcode déposé en pile (Unité 2) ne s'exécute plus. On le voit dans l'en-tête `GNU_STACK` :

RÉEL — READELF -LW (RWE = EXÉCUTABLE, RW = NX)

```
# binaire de labo (protections desactivees)
GNU_STACK  ... RWE      <- pile executable (NX off)
__stack_chk_fail : 0    <- pas de canary
# binaire durci
GNU_STACK  ... RW       <- pile non executable (NX on)
__stack_chk_fail : 1    <- canary present
```

Contournement : ne pas injecter de code, mais **réutiliser le code existant** — ret-to-libc et ROP (§6).

4. ASLR

Les adresses (pile, tas, bibliothèques) sont **randomisées** à chaque exécution : l'attaquant ne connaît plus les adresses à viser.

RÉEL — BASE DE LA LIBC SUR DEUX EXÉCUTIONS (ASLR ACTIVÉ)

```
run 1 : libc @ 0xf7cd8000
run 2 : libc @ 0xf7d45000      <- adresse differente a chaque execution
```

Contournement : fuite d'information (révéler une adresse pour recalculer la base), ou force brute en 32 bits (entropie faible).

5. PIE & RELRO

PIE randomise aussi l'adresse de base de l'exécutable lui-même (pas seulement les bibliothèques). **RELRO** (Relocation Read-Only) rend la table `GOT` non inscriptible après l'édition de liens, bloquant les détournements par écrasement d'entrées GOT. Vérifiables avec `checksec` / `readelf`.

6. Contourner NX : ret-to-libc et ROP

Ret-to-libc (meltypot) : au lieu d'un shellcode, on fait « retourner » l'exécution dans une fonction de la libc déjà présente et exécutable, p. ex. `system("/bin/sh")`, en plaçant sur la pile l'adresse de `system` et son argument.

ROP (Return-Oriented Programming) généralise l'idée : on enchaîne de petits fragments de code existant se terminant par `ret` (les **gadgets**) pour composer n'importe quel calcul, sans jamais injecter de code. On trouve les gadgets avec un outil comme `ropper` :

RÉEL — GADGETS TROUVÉS PAR ROPPER DANS UN BINAIRE DE LABO

```
0x080490f2: add esp, 0x10; leave; ret;
0x0804901b: add esp, 8; pop ebx; ret;
0x08049193: add esp, 4; nop; leave; ret;
0x0804901e: pop ebx; ret;
```

SCHÉMA — CHAÎNE ROP

```
  [ adr. retour ] -> gadget1 (pop eax; ret) -> gadget2 (pop ebx; ret) -> ... -> syscall
                     \_____ chaine ROP : des ret qui s'enchainent _____/
```

7. Tableau de synthèse

Protection	Empêche	Contournement
Stack canary	écrasement de l'adresse de retour via débordement linéaire	fuite + réécriture du canari ; saut par-dessus
NX / DEP	exécution de shellcode en pile/tas	ret-to-libc, ROP
ASLR	connaissance des adresses	fuite d'info, force brute (32 bits)
PIE / RELRO	base fixe de l'exe ; écrasement GOT	fuite d'info ; contournements avancés

Aucune défense n'est suffisante seule : c'est leur **combinaison** (défense en profondeur) qui rend l'exploitation coûteuse — et les langages sûrs en mémoire (Unité 6) qui l'éliminent à la source.

8. Points clés

- Canary, NX, ASLR, PIE/RELRO : chacun bloque une étape de l'exploitation de l'Unité 2.
- Chacun a un contournement connu : fuite d'info, ret-to-libc, ROP.
- La robustesse vient de leur combinaison, pas d'une protection isolée.

Défenses issues de meltypot ; ROP/PIE/RELRO auteur. Sorties canary/readelf/ASLR/ropper produites et vérifiées.